

Query Optimization: Transformation of Relational Expressions — DBMS

Definition: Query Optimization is the component of a Database Management System (DBMS) that attempts to determine the most efficient execution plan for evaluating a given query by transforming relational algebra expressions into equivalent, lower-cost evaluation trees.

1. Detailed Technical Explanation

Query Processing Steps:

```
High-Level SQL Query
    |
    v
[ Parser & Translator ] ---> Generates Relational Algebra Expression Tree
    |
    v
[ Optimizer Engine ]    ---> Applies Equivalence Rules & Cost Formulas
    |
    v
[ Execution Engine ]    ---> Runs Physical Evaluation Plan on Disk Blocks
```

Key Relational Algebra Equivalence Rules:

1. Commutativity of Selection:

- $\sigma_{\theta_1}(\sigma_{\theta_2}(E)) \equiv \sigma_{\theta_2}(\sigma_{\theta_1}(E))$

2. Cascading of Selection:

- $\sigma_{\theta_1 \wedge \theta_2}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E))$

3. Commutativity of Join:

- $E_1 \bowtie_{\theta} E_2 \equiv E_2 \bowtie_{\theta} E_1$

4. Associativity of Join:

- $(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$

5. Pushing Selections Down Trees:

- Perform selection operations (σ) as early as possible before joins ($\bowtie$) to reduce intermediate table size.

Heuristic Optimization Algorithm:

1. Break down complex query conditions into simple selections.
2. **Push Selections Down:** Move σ down the query tree towards leaf nodes.
3. **Push Projections Down:** Move π down to keep only required attributes.
4. Replace Cartesian products followed by selections ($\sigma(\times)$) with Join operations ($\bowtie$).

2. Core Concepts & Memory Keywords

- **Equivalence Rules:** Algebraic identities ensuring two query trees return identical tuple results.
 - **Pushing Selections:** Filtering rows early to minimize intermediate memory usage.
 - **Heuristic Optimization:** Rule-based query tree transformation.
-

3. Must-Write Points for Exams

- Query optimization chooses the physical execution path with minimal disk I/O cost.
 - Pushing selection operators down relational trees dramatically reduces intermediate table sizes before joins.
 - Applying equivalence rules guarantees that equivalent relational expressions yield identical query results.
-

4. Quick Recall Flow

SQL Query -> Parse Relational Tree -> Push Selections Down -> Replace Cartesian Product with Join -> Optimal Execution Plan